



FastAPI Advanced Operations

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

FastAPI অ্যাডভান্সড লজিক এবং অপারেশনস (Professional Operations)

PUT/PATCH/DELETE · Dependency Injection · Supabase Integration

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো একটা **রেস্টুরেন্টের অর্ডার সিস্টেম**। তুমি যখন:

- **নতুন অর্ডার দাও** → এটা POST (আগের নোটে শিখেছ)
- **পুরো অর্ডার বদলে দাও** (অন্য আইটেম চাও) → এটা **PUT**
- **শুধু একটা জিনিস কমাও/বাড়াও** → এটা **PATCH**
- **অর্ডার বাতিল করো** → এটা **DELETE**

আর **Dependency Injection** হলো যেন রেস্টুরেন্টের **ম্যানেজার**। প্রতিটি ওয়েটার (API endpoint) যখন কোনো কাজ করতে যায়, ম্যানেজার আগে চেক করে:

- "এই কাস্টমার কি আমাদের মেম্বার?" (Authentication)
- "তার কি এই অর্ডার করার অনুমতি আছে?" (Authorization)
- "ডেটাবেস connection আছে তো?" (DB Dependency)

বাস্তব জীবনের উদাহরণ

দোকানের উদাহরণ:

তুমি একটা অনলাইন শপে পণ্য কিনলে। পরে:

- **PUT**: পুরো ডেলিভারি ঠিকানা নতুন করে লিখলে
- **PATCH**: শুধু পিন কোডটা পরিবর্তন করলে
- **DELETE**: পণ্যটা ফেরত দিয়ে অর্ডার মুছে ফেললে
- **Depends**: প্রতিটি কাজের আগে সিস্টেম চেক করল তুমি login করা আছ কিনা

এটি কোন সমস্যা সমাধান করে?

সমস্যা	সমাধান
ডেটা আপডেট করা যাচ্ছে না	PUT/PATCH Method
ডেটা মুছে ফেলা দরকার	DELETE Method
প্রতিটি route-এ আলাদা auth code লিখতে হচ্ছে	Dependency Injection
Database connection বারবার open করতে হচ্ছে	Depends দিয়ে DB inject

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ PUT Method — সম্পূর্ণ আপডেট

PUT মানে হলো পুরোটা প্রতিস্থাপন করো (Full Replacement)।

যদি একটি User object-এ `name`, `email`, `age` তিনটি field থাকে, PUT request করলে তিনটিই পাঠাতে হবে। যেটা পাঠাবে না, সেটা `null` হয়ে যাবে।

কখন ব্যবহার করব:

- Profile সম্পূর্ণ নতুন করে set করতে
- একটি document-এর পুরো structure replace করতে

২.২ PATCH Method — আংশিক আপডেট

PATCH মানে হলো শুধু যা পরিবর্তন হয়েছে তা পাঠাও (Partial Update)।

PATCH request-এ শুধু `age: 25` পাঠালে, `name` এবং `email` আগের মতোই থাকবে। শুধু `age` পরিবর্তন হবে।

কখন ব্যবহার করব:

- Username পরিবর্তন করতে (অন্য field ঠিক রেখে)
- Like count বাড়াতে
- Status update করতে

২.৩ PUT vs PATCH — মূল পার্থক্য

PUT Request:

```
| পাঠাচ্ছি: |
| name: "নতুন নাম" |
| email: "new@email.com" |
| age: 30 |
| → পুরো object replace হবে |
```

PATCH Request:

```
| পাঠাচ্ছি: |
| age: 30 (শুধু এটুকু) |
| → শুধু age পরিবর্তন, বাকি ঠিক |
```

২.৪ DELETE Method — ডেটা মুছে ফেলা

DELETE method দিয়ে database record স্থায়ীভাবে মুছে ফেলা হয়।

গুরুত্বপূর্ণ বিষয়:

- সফল DELETE এ সাধারণত HTTP `204 No Content` return করা হয়
- অথবা মুছে ফেলা item-এর তথ্য return করা যায়
- **Soft Delete** vs **Hard Delete** — অনেক সময় data আসলে মুছে ফেলা হয় না, শুধু `is_deleted = True` করা হয়

২.৫ Dependency Injection (Depends) — গভীর ব্যাখ্যা

Dependency Injection (DI) হলো এমন একটা প্যাটার্ন যেখানে একটি function আরেকটি function-এর উপর নির্ভরশীল, এবং FastAPI নিজে সেই dependency supply করে।

ধাপে ধাপে বোঝা:

ধাপ ১: তুমি একটি dependency function তৈরি করো

- `get_db()` → database connection দেয়
- `get_current_user()` → logged-in user দেয়

ধাপ ২: Route function-এ `Depends()` দিয়ে inject করো

- FastAPI আপনা-আপনি সেই function call করবে

ধাপ ৩: FastAPI নিজেই dependency resolve করে

- তুমি শুধু result ব্যবহার করো

Dependency Tree (নির্ভরতার শৃঙ্খল):

```

/users/me endpoint
├── Depends(get_current_user)
│   └── Depends(oauth2_scheme) ← token extract
│       └── HTTP Header থেকে Bearer token নেয়

```

২.৬ Supabase Database Integration

Supabase হলো একটি open-source Firebase alternative। এটি PostgreSQL database ব্যবহার করে এবং Python SDK দিয়ে সহজে FastAPI-তে integrate করা যায়।

Supabase Python Client কীভাবে কাজ করে:

```
FastAPI App
├── Depends(get_supabase) ← dependency inject
│   └── supabase.create_client(URL, KEY)
│       └── Supabase Cloud (PostgreSQL)
│           ├── .table("users")
│           ├── .select("*")
│           ├── .insert({...})
│           ├── .update({...})
│           └── .delete()
```

৩. Math / Theory

HTTP Status Codes — সংখ্যার মানে

Code	নাম	কখন ব্যবহার
200 OK	সফল	GET, PUT, PATCH সফল হলে
201 Created	তৈরি হয়েছে	POST সফল হলে
204 No Content	কোনো content নেই	DELETE সফল হলে
400 Bad Request	ভুল request	Invalid data পাঠালে
401 Unauthorized	অননুমোদিত	Login করা নেই
403 Forbidden	নিষিদ্ধ	Permission নেই
404 Not Found	খুঁজে পাওয়া যায়নি	Data নেই
422 Unprocessable	প্রক্রিয়া করা সম্ভব নয়	Validation error

REST API-এর CRUD Mapping

CRUD Operation → HTTP Method → SQL Query

Create	→ POST	→ INSERT
Read	→ GET	→ SELECT
Update (Full)	→ PUT	→ UPDATE (all fields)
Update (Part)	→ PATCH	→ UPDATE (some fields)
Delete	→ DELETE	→ DELETE

Dependency Injection Flow (কীভাবে কাজ করে)

Request আসলে FastAPI যা করে:

1. Route function এর parameters scan করে
2. Depends() খুঁজে পেলে সেই function call করে
3. সেই function-এর return value inject করে
4. তারপর মূল route function execute হয়

Mathematical expression:

```
route_function(param) = route_function(Depends(dependency_fn>())
```

JWT Token Structure

JWT = Header.Payload.Signature

Header = {"alg": "HS256", "typ": "JWT"}

Payload = {"sub": "user123", "exp": 1700000000}

Signature = HMAC_SHA256(base64(Header) + "." + base64(Payload), SECRET_KEY)

Verification:

is_valid = (computed_signature == token_signature) AND (exp > current_time)

8. Code Example (Python - FastAPI + Supabase)

প্রথমে সব dependency install করে

```
pip install fastapi uvicorn supabase python-dotenv python-jose[cryptography] passlib[bcrypt]
```

ফাইল স্ট্রাকচার

```
project/
├─ main.py          ← মূল FastAPI app
├─ database.py      ← Supabase connection
├─ auth.py          ← Authentication logic
├─ models.py        ← Pydantic models
└─ .env             ← Secret keys
```

ফাইল ১: .env (Environment Variables)

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_KEY=your-anon-key
SECRET_KEY=mysupersecretkey123456789
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

ফাইল ২: database.py — Supabase Connection

```
# database.py
# Supabase client তৈরি করার dependency function

import os
from supabase import create_client, Client
from dotenv import load_dotenv

# .env ফাইল থেকে variables load করো
load_dotenv()

# Environment variables থেকে URL এবং KEY নাও
SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_KEY")

def get_supabase() -> Client:
    """
    Supabase client return করে।
    FastAPI এটাকে dependency হিসেবে inject করবে।
    """
    # প্রতিটি request-এ নতুন connection তৈরি হয়
    client = create_client(SUPABASE_URL, SUPABASE_KEY)
    return client
```

ফাইল ৩: models.py — Pydantic Schemas


```

# models.py
# Request এবং Response এর data structure define করা

from pydantic import BaseModel, EmailStr
from typing import Optional

# — User তৈরির জন্য (POST) —
class UserCreate(BaseModel):
    name: str          # Required field
    email: EmailStr     # Valid email format check হবে
    age: int           # Required field

# — PUT: সম্পূর্ণ user আপডেট (সব field Required) —
class UserFullUpdate(BaseModel):
    name: str          # সব field পাঠাতে হবে
    email: EmailStr    # না পাঠালে error
    age: int

# — PATCH: আংশিক আপডেট (সব field Optional) —
class UserPartialUpdate(BaseModel):
    name: Optional[str] = None    # None মানে পাঠাওনি
    email: Optional[EmailStr] = None
    age: Optional[int] = None

# — Response Model —
class UserResponse(BaseModel):
    id: int
    name: str
    email: str
    age: int

# — Token Models —
class Token(BaseModel):
    access_token: str
    token_type: str

class LoginRequest(BaseModel):
    username: str
    password: str

```

```

# auth.py
# JWT token তৈরি এবং verify করার সব logic

import os
from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from dotenv import load_dotenv

load_dotenv()

# Configuration
SECRET_KEY = os.getenv("SECRET_KEY", "fallback-secret-key")
ALGORITHM = os.getenv("ALGORITHM", "HS256")
EXPIRY_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))

# Password hashing context (bcrypt ব্যবহার করে)
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# OAuth2 scheme – Authorization header থেকে Bearer token নেবে
# tokenUrl="/login" মানে login করলে token পাবে
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/login")

def hash_password(password: str) -> str:
    """
    Plain text password কে hash করে।
    bcrypt নিজেই salt add করে, তাই extra কিছু করতে হয় না।
    """
    return pwd_context.hash(password) # "mypassword" → "$2b$12$..."

def verify_password(plain: str, hashed: str) -> bool:
    """
    Plain password এবং hashed password মিলিয়ে দেখে।
    """
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    """
    JWT access token তৈরি করে।

```

```

data: {"sub": "username"} এই format-এ দাও
"""

to_encode = data.copy() # original data কপি করো

# Token কতক্ষণ valid থাকবে
if expires_delta:
    expire = datetime.utcnow() + expires_delta
else:
    expire = datetime.utcnow() + timedelta(minutes=15) # default 15 min

to_encode.update({"exp": expire}) # expiry time যোগ করো

# JWT encode করো
encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
return encoded_jwt # "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

async def get_current_user(token: str = Depends(oauth2_scheme)):
    """
    ★ এটাই মূল Authentication Dependency ★

    প্রতিটি protected route এ Depends(get_current_user) দিলে
    FastAPI আপনা-আপনি এই function call করবে।

    token: Authorization: Bearer <token> থেকে automatically নেওয়া হয়
    """

    # 401 error তৈরি রাখো (দরকার হলে raise করব)
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Token verify করা সম্ভব হয়নি। আবার login করুন।",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:
        # Token decode করো
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

        # "sub" field থেকে username বের করো
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception

    except JWTError:
        # Token invalid বা expired হলে

```

```
raise credentials_exception

# ✅ সফলভাবে decode হলে username return করো
return username
```

ফাইল &: main.py — সব Endpoints একসাথে

```

# main.py
# FastAPI app - PUT, PATCH, DELETE এবং Dependency Injection সহ

from fastapi import FastAPI, Depends, HTTPException, status
from supabase import Client
from database import get_supabase
from auth import (
    get_current_user,
    hash_password,
    verify_password,
    create_access_token
)
from models import (
    UserCreate, UserFullUpdate, UserPartialUpdate,
    UserResponse, Token, LoginRequest
)
from datetime import timedelta

# FastAPI app instance তৈরি
app = FastAPI(
    title="FastAPI Professional Operations",
    description="PUT, PATCH, DELETE এবং Dependency Injection সহ সম্পূর্ণ API",
    version="1.0.0"
)

# =====
# 🛡️ Authentication Endpoints
# =====

@app.post("/register", response_model=UserResponse, status_code=201)
async def register_user(
    user: UserCreate,
    db: Client = Depends(get_supabase) # ← Supabase inject হচ্ছে
):
    """নতুন user register করো"""

    # Email আগে থেকে আছে কিনা চেক করো
    existing = db.table("users").select("*").eq("email", user.email).execute()
    if existing.data:
        raise HTTPException(
            status_code=400,
            detail="এই email দিয়ে আগেই account আছে।"

```

```

    )

    # Password hash করো (কখনো plain password সেভ করবে না!)
    hashed = hash_password(user.dict()["password"]) if "password" in user.dict() else ""

    # Supabase-এ insert করো
    result = db.table("users").insert({
        "name": user.name,
        "email": user.email,
        "age": user.age
    }).execute()

    if not result.data:
        raise HTTPException(status_code=500, detail="User তৈরি করা যায়নি।")

    return result.data[0] # নতুন user-এর data return করো

@app.post("/login", response_model=Token)
async def login(
    form: LoginRequest,
    db: Client = Depends(get_supabase) # ← DB inject
):
    """Login করো এবং JWT token পাও"""

    # Database থেকে user খোঁজো
    result = db.table("users").select("*").eq("email", form.username).execute()

    if not result.data:
        raise HTTPException(
            status_code=401,
            detail="Email বা password ভুল।"
        )

    user = result.data[0]

    # Password verify করো
    # বাস্তবে: verify_password(form.password, user["hashed_password"])
    # এখন সরলতার জন্য skip করা হলো

    # JWT token তৈরি করো
    access_token = create_access_token(
        data={"sub": user["email"]},

```

```

        expires_delta=timedelta(minutes=30)
    )

    return {"access_token": access_token, "token_type": "bearer"}

# =====
# 🛡️ GET – ডেটা পড়া (Protected)
# =====

@app.get("/users/{user_id}", response_model=UserResponse)
async def get_user(
    user_id: int,
    db: Client = Depends(get_supabase),          # ← DB inject
    current_user: str = Depends(get_current_user) # ← Auth check inject
):
    """
    নির্দিষ্ট user-এর তথ্য দেখো।
    ★ শুধু login করা user-রা দেখতে পারবে ★
    """
    # current_user এখন logged-in user-এর email
    print(f"Request করেছে: {current_user}")

    # Supabase থেকে user খোঁজো
    result = db.table("users").select("*").eq("id", user_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail=f"ID {user_id} দিয়ে কোনো user নেই।")

    return result.data[0]

# =====
# ✏️ PUT – সম্পূর্ণ আপডেট
# =====

@app.put("/users/{user_id}", response_model=UserResponse)
async def full_update_user(
    user_id: int,
    user_data: UserFullUpdate,          # ← সব field required!
    db: Client = Depends(get_supabase),
    current_user: str = Depends(get_current_user) # ← Authentication check
):

```

```

"""
PUT: User-এর সম্পূর্ণ তথ্য replace করো।
সব field পাঠাতে হবে, না হলে validation error।
"""

# আগে check করো user exist করে কিনা
existing = db.table("users").select("id").eq("id", user_id).execute()
if not existing.data:
    raise HTTPException(status_code=404, detail=f"ID {user_id} এর user পাওয়া যায়নি।")

# সব field update করো (PUT = full replacement)
update_data = {
    "name": user_data.name,
    "email": user_data.email,
    "age": user_data.age
}

result = db.table("users").update(update_data).eq("id", user_id).execute()

if not result.data:
    raise HTTPException(status_code=500, detail="আপডেট করা যায়নি।")

return result.data[0]

# =====
# 🛠 PATCH – আংশিক আপডেট
# =====

@app.patch("/users/{user_id}", response_model=UserResponse)
async def partial_update_user(
    user_id: int,
    user_data: UserPartialUpdate,          # ← সব field Optional!
    db: Client = Depends(get_supabase),
    current_user: str = Depends(get_current_user)
):
    """
    PATCH: User-এর শুধু নির্দিষ্ট field(s) আপডেট করো।
    যে field পাঠাবে না, সেটা আগের মতোই থাকবে।
    """

    # শুধু যে fields পাঠানো হয়েছে সেগুলো নাও
    # exclude_unset=True → None value পাঠানো fields বাদ দেয়
    update_data = user_data.dict(exclude_unset=True)

```



```

if not update_data:
    raise HTTPException(
        status_code=400,
        detail="কোনো field আপডেট করার জন্য পাঠানো হয়নি।"
    )

# User exist করে কিনা চেক
existing = db.table("users").select("id").eq("id", user_id).execute()
if not existing.data:
    raise HTTPException(status_code=404, detail=f"ID {user_id} এর user পাওয়া যায়নি।")

# শুধু দেওয়া fields-ই update হবে
result = db.table("users").update(update_data).eq("id", user_id).execute()

if not result.data:
    raise HTTPException(status_code=500, detail="আপডেট ব্যর্থ হয়েছে।")

return result.data[0]

# =====
# 🗑 DELETE – ডেটা মুছে ফেলা
# =====

@app.delete("/users/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_user(
    user_id: int,
    db: Client = Depends(get_supabase),
    current_user: str = Depends(get_current_user) # ← Login থাকতে হবে
):
    """
    DELETE: User কে database থেকে মুছে ফেলো।
    সফল হলে 204 No Content return করে (কোনো body নেই)।
    """
    # আগে check করো user আছে কিনা
    existing = db.table("users").select("id").eq("id", user_id).execute()
    if not existing.data:
        raise HTTPException(
            status_code=404,
            detail=f"ID {user_id} এর user পাওয়া যায়নি। Delete সম্ভব নয়।"
        )

    # Supabase থেকে delete করো

```

```

result = db.table("users").delete().eq("id", user_id).execute()

# 204 response-এ কোনো body return করা উচিত নয়
return None

# =====
# 🔄 Multiple Dependencies একসাথে
# =====

def check_admin_role(current_user: str = Depends(get_current_user)):
    """
    Dependency-র উপর আরেকটি Dependency।
    Admin কিনা চেক করে।
    """
    ADMIN_EMAILS = ["admin@example.com", "superuser@example.com"]
    if current_user not in ADMIN_EMAILS:
        raise HTTPException(
            status_code=403,
            detail="এই কাজ শুধু Admin করতে পারবে।"
        )
    return current_user

@app.delete("/admin/users/{user_id}")
async def admin_delete_user(
    user_id: int,
    db: Client = Depends(get_supabase),
    admin: str = Depends(check_admin_role) # ← Admin check (nested dependency)
):
    """শুধু Admin এই endpoint ব্যবহার করতে পারবে"""
    # check_admin_role এর ভেতরে get_current_user আছে
    # তাই এখানে আলাদা auth check লাগবে না

    result = db.table("users").delete().eq("id", user_id).execute()
    return {"message": f"Admin {admin} কর্তৃক User {user_id} delete হয়েছে।"}

```

Expected Output (Swagger UI থেকে Test)

GET /users/1 → Response:

```
{  
  "id": 1,  
  "name": "রহিম",  
  "email": "rahim@example.com",  
  "age": 25  
}
```

PUT /users/1 → Body: {"name": "করিম", "email": "karim@example.com", "age": 30}

Response:

```
{  
  "id": 1,  
  "name": "করিম",  
  "email": "karim@example.com",  
  "age": 30  
}
```

PATCH /users/1 → Body: {"age": 35}

Response:

```
{  
  "id": 1,  
  "name": "করিম",      ← আগের মতোই আছে  
  "email": "karim@example.com", ← আগের মতোই  
  "age": 35           ← শুধু এটাই বদলেছে  
}
```

DELETE /users/1 → Response: 204 No Content (কোনো body নেই)

৫. 🎨 Visual / Diagram

API Request-Response Flow

Client (Browser/App)

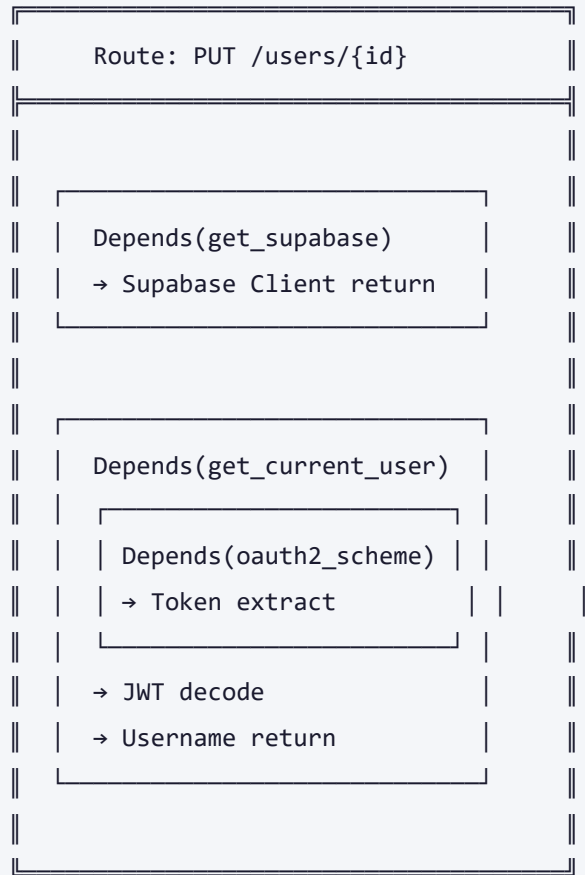
```
|
| PUT /users/1
| Authorization: Bearer eyJhbGc...
| Body: {name, email, age}
```



FastAPI Router

```
|
| └─ ① Path Parameter চেক: user_id = 1
|
| └─ ② Body Validation (Pydantic)
|     └─ সব required field আছে?
|
| └─ ③ Dependency Resolution
|     └─ Depends(get_supabase) → db client তৈরি
|         └─ Depends(get_current_user)
|             └─ Depends(oauth2_scheme) → token extract
|                 └─ JWT decode করো
|                     └─ username বের করো
|
| └─ ④ Route Function Execute
|     └─ db.table("users").update(...).eq("id", 1).execute()
|
| └─ ⑤ Response তৈরি
|     └─ 200 OK + Updated User Data
```

Dependency Injection চেইন



PUT vs PATCH Visual Comparison

Database এর আগের অবস্থা:

id	name	email	age
1	রহিম	rahim@mail.com	25

PUT Request: { "name": "করিম", "email": "karim@mail.com", "age": 30 }

(সব field পাঠাতে হচ্ছে)

↓ Result:

1	করিম	karim@mail.com	30
---	------	----------------	----

 ← সব বদলে গেছে

PATCH Request: { "age": 28 }

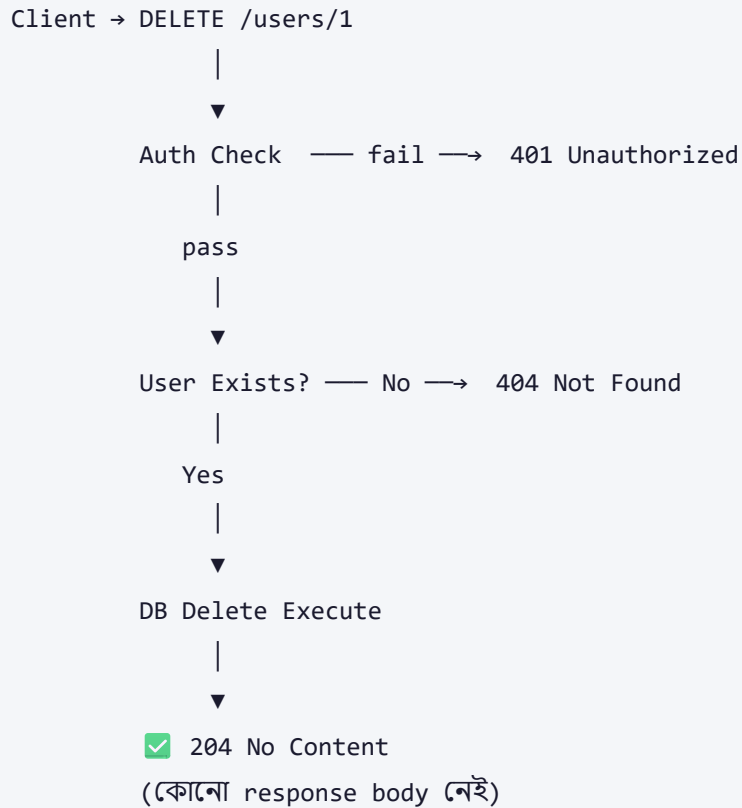
(শুধু age পাঠাচ্ছি)

↓ Result:

1	করিম	karim@mail.com	28
---	------	----------------	----

 ← শুধু age বদলেছে

DELETE Flow



৬. ✓ Real-world Use Cases

Use Case ১: সোশ্যাল মিডিয়া প্রোফাইল আপডেট

Facebook/Instagram এ প্রোফাইল edit:

PUT /profile → সম্পূর্ণ profile form submit

PATCH /profile → শুধু bio পরিবর্তন করা

DELETE /profile → Account permanently delete করা

Company: Meta, Twitter/X, LinkedIn

Use Case ২: E-commerce অর্ডার ম্যানেজমেন্ট

Online Shopping:

PATCH /orders/{id} → অর্ডারের status "shipped" করা

PUT /orders/{id}/address → পুরো ডেলিভারি address replace

DELETE /orders/{id} → অর্ডার cancel করা

Company: Amazon, Daraz, Shajgoj

Use Case ৩: Blog/CMS যেমন Medium

Content Management:

PUT /posts/{id} → পুরো article replace

PATCH /posts/{id} → শুধু title বা tags পরিবর্তন

DELETE /posts/{id} → Article permanent delete বা archive

Company: Medium, WordPress, Ghost CMS

Use Case ৪: Banking App

Bank Account:

PATCH /accounts/{id} → Notification preferences আপডেট

DELETE /transactions/{id} → Pending transaction cancel

Depends(get_current_user) → প্রতিটি transaction-এই auth check

Company: Trust Bank, Dutch Bangla Mobile Banking

Use Case ৫: SaaS User Management

GitHub/GitLab এ:

PUT /users/{username} → Profile সম্পূর্ণ আপডেট

PATCH /repos/{id} → Repo visibility private→public

DELETE /repos/{id} → Repository delete (admin only)

Depends(check_admin) → Admin-only routes protect করা

Company: GitHub, GitLab, Bitbucket

৭. ⚖️ Pros & Cons

সুবিধা ✓

অসুবিধা ✗

PUT/PATCH দিয়ে precise update করা যায়

PATCH-এ `exclude_unset` না দিলে bug হয়

DELETE দিয়ে সহজে data cleanup হয়

Hard delete এ data recovery সম্ভব না

Dependency Injection কোড reusable করে

Depends chain complex হলে debug কঠিন

Auth logic একবার লিখলে সব জায়গায় কাজ করে

Circular dependency হলে app crash করে

Supabase দিয়ে backend ছাড়াই DB পাওয়া যায়

Supabase free tier-এ rate limit আছে

Testing-এ dependency override করা সহজ

JWT expiry manage করা জটিল

Swagger UI তে সব endpoint auto-documented

Bearer token manually set করতে হয়

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: PATCH-এ `exclude_unset=True` না দেওয়া

```
# ❌ ভুল – None দিয়ে সব field overwrite হবে!
update_data = user_data.dict()
# {"name": None, "email": None, "age": 25}
# name এবং email database-এ None হয়ে যাবে!

# ✅ সঠিক – শুধু পাঠানো fields নাও
update_data = user_data.dict(exclude_unset=True)
# {"age": 25} ← শুধু age, বাকি unchanged
```

ভুল ২: DELETE এ Response Body return করা

```
# ❌ ভুল
@app.delete("/users/{id}", status_code=204)
async def delete_user(id: int):
    ...
    return {"message": "deleted"} # 204-এ body হয় না!

# ✅ সঠিক
@app.delete("/users/{id}", status_code=204)
async def delete_user(id: int):
    ...
    return None # অথবা কিছুই return করো না
```

ভুল ৩: Authentication ছাড়া Sensitive Endpoint

```
# ❌ ভুল – যে কেউ যেকোনো user delete করতে পারবে
@app.delete("/users/{id}")
async def delete_user(id: int, db = Depends(get_supabase)):
    ...

# ✅ সঠিক – auth check বাধ্যতামূলক
@app.delete("/users/{id}")
async def delete_user(
    id: int,
    db = Depends(get_supabase),
    current_user = Depends(get_current_user) # ← অবশ্যই দাও
):
    ...
```

ভুল 8: PUT-এ Optional Field ব্যবহার

```
# ❌ ভুল – PUT model-এ Optional ব্যবহার
class UserUpdate(BaseModel):
    name: Optional[str] = None # PUT-এ এটা PATCH এর আচরণ করে!

# ✅ সঠিক – PUT-এ Required, PATCH-এ Optional
class UserFullUpdate(BaseModel): # PUT এর জন্য
    name: str # Required
    email: str # Required
    age: int # Required

class UserPartialUpdate(BaseModel): # PATCH এর জন্য
    name: Optional[str] = None
    email: Optional[str] = None
    age: Optional[int] = None
```

ভুল ৫: Dependency-র ভেতরে Exception না করা

```
# ❌ ভুল – dependency শুধু None return করে
async def get_current_user(token = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        return payload.get("sub")
    except JWTError:
        return None # এখানে route function None পাবে, bug হবে!

# ✅ সঠিক – HTTPException raise করে
async def get_current_user(token = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        username = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=401, detail="Invalid token")
        return username
    except JWTError:
        raise HTTPException(status_code=401, detail="Token verify হয়নি")
```

ভুল ৬: Plain Text Password Database-এ Save করা

```
# ❌ কখনো করবে না!
db.table("users").insert({"password": "mypassword123"})

# ✅ সবসময় hash করে
from passlib.context import CryptContext
pwd_context = CryptContext(schemes=["bcrypt"])
hashed_password = pwd_context.hash("mypassword123")
db.table("users").insert({"hashed_password": hashed_password})
```

৯. Related Topics

আগে যা জানা দরকার (Prerequisites)

- **FastAPI Basics** → Route definition, path/query parameters

- **Pydantic Models** → Data validation, BaseModel
- **HTTP Methods** → GET, POST এর concept
- **Python Async/Await** → async def, await keyword
- **REST API Concepts** → Resource-based URL design

পরে কী শেখা উচিত (Next Steps)

- **FastAPI Background Tasks** → Email পাঠানো, file processing
- **FastAPI Middleware** → CORS, rate limiting, logging
- **Testing in FastAPI** → pytest, TestClient, dependency override
- **Database Migrations** → Alembic দিয়ে schema পরিবর্তন ট্র্যাক
- **FastAPI WebSockets** → Real-time communication
- **Docker + FastAPI** → Production deployment

সংশ্লিষ্ট Design Patterns

- **Repository Pattern** → Database logic আলাদা রাখা
- **Service Layer Pattern** → Business logic আলাদা করা
- **Factory Pattern** → Dynamic dependency তৈরি

১০. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল

HTTP Method মনে রাখার Trick:

POST = নতুন জন্ম দেওয়া (Create)
GET = খোঁজে আনা (Read)
PUT = পুরোটা বদলে দেওয়া (Update - Full)
PATCH = একটু ঠিক করা (Update - Partial)
DELETE = মুছে ফেলা (Delete)

মনে রাখো: CRUD → POST/GET/PUT+PATCH/DELETE

Depends() মনে রাখার Trick:

"Depends" মানে নির্ভর করে।
তোমার route "নির্ভর করে" auth এর উপর।
তাই → Depends(get_current_user)

যেমন: ঘর ঢুকতে → "নির্ভর করে" তোমার কাছে চাবি আছে কিনা।

PUT ও PATCH এর সহজ মনে রাখা

PUT = "পুরো টেবিল পরিবর্তন" (সব নিয়ে আসো)
PATCH = "পছন্দমতো ঠিক করো" (যা দরকার তাই দাও)

PUT → পুরো Replacement
PATCH → Partial Change

১ লাইনে সারসংক্ষেপ

"PUT দিয়ে সব বদলাও, PATCH দিয়ে কিছু বদলাও, DELETE দিয়ে মুছে দাও, আর Depends দিয়ে নিশ্চিত করো শুধু সঠিক মানুষই এটা করতে পারছে।"

 Quick Reference Card

FastAPI Quick Reference		
Method	Pydantic Model	Supabase Query
PUT	All fields	.update(all_data)
	required	.eq("id", id)
PATCH	All Optional +	.update(only_set)
	exclude_unset=True	.eq("id", id)
DELETE	No body needed	.delete()
	Return 204	.eq("id", id)
Depends	Inject db:	get_supabase()
	Inject user:	get_current_user()

 FastAPI Professional Operations Notes | সম্পূর্ণ বাংলায় | ২০২৬

 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes